



Faculty of Engineering & Technology

Electrical & Computer Engineering Department

REAL-TIME SYSTEM AND INTERFACING

TECHNIQUES LABORATORY

Report #2

MBED Module

Prepared by:

Yasmeen Kamal	1201146
Lana Batnaj	1200308
Aya R Dahbour	1201738

Instructor: Dr. Hanna Bullata

Teaching Assistant : Eng. Rania Shahwan

Section : 1

Date: 24/11/2024

Abstract

This lab module explains the MBED microcontroller system in a simple and practical way, highlighting its key uses in IoT development, education, and rapid prototyping. It focuses on hands-on learning, covering both digital and analog tasks. Applications include basic projects like blinking an LED and displaying data on an LCD, as well as more advanced tasks such as building a binary counter, connecting sensors, and handling interrupts. The MBED ARM online simulator makes it easy to test and debug code virtually, especially when physical resources are limited. The system is user-friendly with ready-to-use C++ libraries and strong community support. It provides an overview of MBED's applications, showing its strengths like wide hardware compatibility and an advanced online development platform, while noting some limitations like library restrictions.

Table of Contents

Abstract	1
1. Theory	1
1.1. MBED Overview	1
1.2. MBED vs Arduino	1
1.3. MBED's advantages and restrictions	1
1.4. Practical Applications of MBED	2
1.4.1. Digital and Analog Signal Processing:	2
1.4.2. MBED Functions of the AnalogIn Class	2
1.5. Timer and Interrupts	2
2. Procedure and Discussion	3
Part 1: Introduction to the MBED environment	3
1.1: Blinking LED	3
Task 1:	3
1.2: LCD Display	4
Part 2: Introduction to ADC, DAC, and Timers	4
2.1: Reading an Analog input	4
2.2: PWM-out	5
2.3: Using Timeout interface	5
2.4: Using Ticker interface	5
3. Conclusion	6
4. References	7
5. Appendix	8
Part 1:	8
1.1: Blinking LED	8
Task1:	8
1.2: LCD Display	9
Part 2:	9
2.1: Reading an Analog input	9
2.2: PWM-OUT:	10
2.3: Using Timeout Interface	10
2.4: Using Ticker Interface	11

Figure 1:MBED	1
Figure 2:Successive approximation ADC block diagram.	2
Figure 3:Blinking LED.....	3
Figure 4:Sequential LED.....	3
Figure 5:Part 2.1	4
Figure 6:Code of appendix 2.1	9

1. Theory

1.1. MBED Overview

MBED is an open-source platform for IoT and 32-bit microcontrollers, built on ARM Cortex-M processors. It offers an online compiler, extensive libraries, security features, connectivity options, real-time operating system (RTOS) support, and pre-built drivers for sensors and devices. Compatible with various hardware boards, MBED is user-friendly, cross-platform, and supported by a strong developer community, making it ideal for robotics and IoT applications. Developers can choose between the web-based MBED Online Compiler and the offline MBED Studio for coding, catering to different needs. [1]

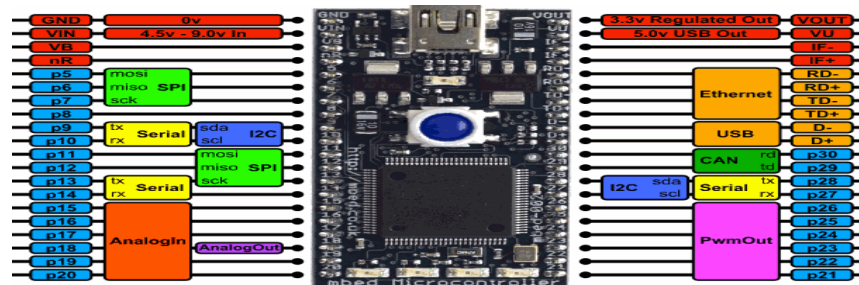


Figure 1:MBED

1.2. MBED vs Arduino

The comparison between MBED and Arduino reveals significant differences in specifications and capabilities. MBED, based on ARM architecture, operates at a clock speed of 96 MHz, while Arduino uses AVR architecture with a clock speed of 16 MHz. In terms of memory, MBED boasts 512 KB of flash memory and 32 KB of RAM, far surpassing Arduino's 32 KB of flash memory and 2 KB of RAM. MBED also excels in network connectivity, offering built-in Ethernet, a feature absent in Arduino. Development environments vary, with MBED utilizing an online compiler and Arduino relying on an offline IDE. Cost-wise, MBED is priced around 70 USD, considerably higher than Arduino's more affordable 3 USD.

Additionally

1.3. MBED's advantages and restrictions

MBED has some good features like being easy to use and helpful for learning. But it also has some drawbacks, such as depending on libraries that limit low-level programming, its libraries not being open-source, and not having specific debugging tools.

1.4. Practical Applications of MBED

1.4.1. Digital and Analog Signal Processing:

MBED demonstrates proficiency in managing both digital and analog signals. It leverages Successive-Approximation Analog-to-Digital Converters (ADC) for accurate signal conversion and utilizes Digital-to-Analog Converters (DAC) to convert digital codes into analog signals. This capability proves vital in various applications, including sensor interfacing and audio processing. [2]

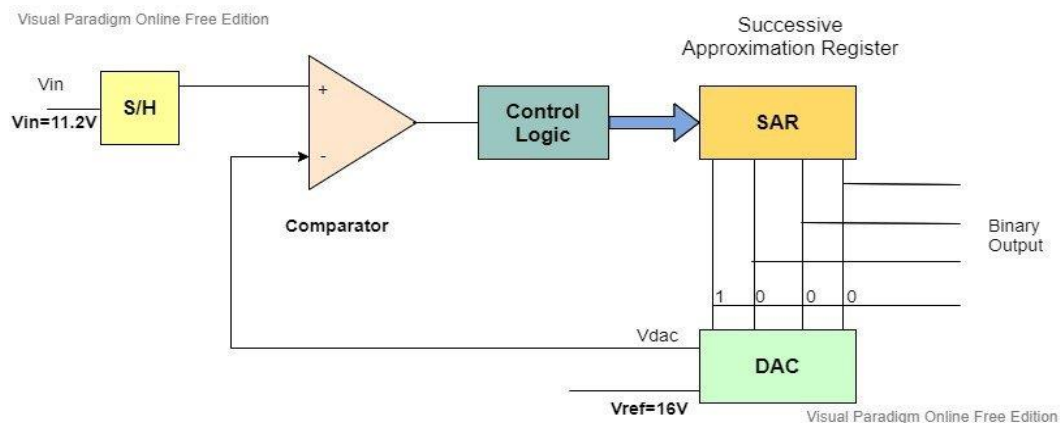


Figure 2: Successive approximation ADC block diagram.

1.4.2. MBED Functions of the AnalogIn Class

The AnalogIn class in MBED is crucial for efficiently managing analog inputs. It allows connections to designated pins and offers customizable reference voltage settings. Users can read input voltages in various formats, such as normalized floating-point, unsigned short, or direct voltage values. This class is vital for precise and adaptable handling of analog signals, ensuring flexibility and accuracy in different applications. [2]

1.5. Timer and Interrupts

Microcontrollers use timers for tasks like delays, counting, generating waveforms, and PWM signals. In ARM MBED, 32-bit timers allow users to set, read, and count time independently, with options to create, start, and stop timers as needed. The maximum time a timer can handle is calculated using the formula: $T_{max} = (2^{31} - 1)$ microseconds, which equals roughly 2147.5 seconds or 35 minutes and 47 seconds. [3]

2. Procedure and Discussion

Part 1: Introduction to the MBED environment

1.1: Blinking LED

This part of the experiment demonstrates how to create a simple program using Mbed to make an LED blink. The code in the appendix is designed to control the built-in LED (LED1) on the Mbed board. The main function runs in an infinite loop, turning LED1 on and off with a 0.5-second delay between each state. This creates a continuous blinking effect, as shown in Figures X and Y below.



Figure 3: Blinking LED

The code creates a digital output object named "led," which is connected to the onboard LED1 pin of the microcontroller to control the LED. The `'while(true)'` loop runs the code inside it continuously to keep the LED blinking. Turn the LED on by setting its state to high (1) and keeping it for 0.5 seconds; then turn it off by setting it to low (0) and keeping it for another 0.5 seconds.

Task 1: Write code to make the four LEDs switch on one after the other. First, LED1 is on for 0.2s, then LED2 is on for 0.2s, and finally, LED3 is on for 0.2s.

This task is to light four LEDs one by one in a cascading effect. The code lights LED1 for 0.2 seconds and turns it off, then turns LED2 on for the same amount of time and turns it off as it turns LED3 on, then LED4. Each of these LEDs stays on for 0.2 seconds before the next LED lights up, as in Figure X. It then just loops to make a smooth continuous pattern.



Figure 4: Sequential LED

1.2: LCD Display

This part involves using the Mbed microcontroller to control an LCD for displaying text. The code creates an object called "lcd" and specifies SPI pins for communication along with two control pins. Inside an infinite while) loop, the LCD cursor is positioned at column 3, row 3 using LCD.locate(3, 3). The message "Write your message" is then displayed with lcd. printf(), and the screen is cleared afterward using lcd.cls().

We're sorry, but we accidentally forgot to take a picture of this part, even though we did complete it and displayed our names on the LCD. We hope you can understand!

Part 2: Introduction to ADC, DAC, and Timers

2.1: Reading an Analog input

The circuit was configured as shown in Figure 4, with the potentiometer connected between +5V and GND and the wiper attached to the LPC11U24 microcontroller's analog input pin (P19). The voltage at the analog input pin was varied by adjusting the resistance of the potentiometer. The microcontroller then reads the input voltage and processes it for future use or display.

<https://drive.google.com/file/d/1knxyvzeSvJoHze8uT1QklYyhqgdngELb/view?usp=sharing>

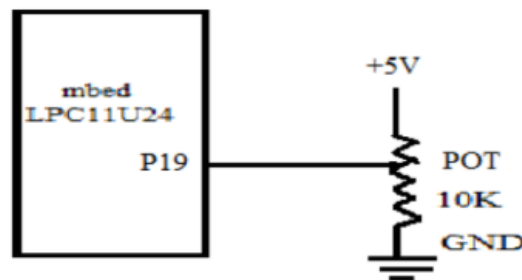


Figure 5:Part 2.1

2.2: PWM-out

The PwmOut interface was used to set the frequency and duty cycle of a PWM signal. In this section, the brightness of LED1 was regulated using PWM. This feature was achieved by writing and compiling a C++ application with the Mbed compiler the code held in Appendix 2.2.

https://drive.google.com/file/d/1t7b4_iAqE-hlsr8K0idz4qRLfbopppxd/view?usp=sharing

2.3: Using Timeout interface

The Timeout interface was used to configure an interrupt that called a function after a predetermined time. A simple software was built to configure a Timeout to reverse the status of an LED after a certain time. The Timeout Produces a single action like a one-time toggle at the LED.

<https://drive.google.com/file/d/15hcdvSWKTJKTS0xfbFfrIPni2Jv1heT-/view?usp=sharing>

2.4: Using Ticker interface

The Ticker interface was used to configure a periodic interrupt that called a function at a predetermined pace. A simple software was built to set up the Ticker to reverse the status of an LED at regular intervals. The Ticker produces recurring output which shows as continuous blinking.

<https://drive.google.com/file/d/17aZCHun-jPuEdhNjsPVAaqnBjVtLYi9p/view?usp=sharing>

3.Conclusion

This experiment provided a clear understanding of the MBED microcontroller and its functionality with the ARM Mbed OS simulator. The MBED platform was explored by comparing it to Arduino to highlight key differences. Focus was placed on the hardware, particularly the number and functionality of its pins. Through practical examples on the simulator, the use of features such as ADC and DAC was demonstrated, along with MBED's ability to handle communication protocols like I2C. The experiment showcased MBED's capabilities in real-time applications and its role in fostering innovation and hands-on learning in technology and engineering.

4. References

- [1] [Online]. Available: <https://os.mbed.com/>. [Accessed 24 11 2024].
- [2] [Online]. Available: <https://www.geeksforgeeks.org/difference-between-analog-and-digital-processing/>. [Accessed 2024 11 24].
- [3] [Online]. Available: <https://www.allaboutcircuits.com/technical-articles/introduction-to-microcontroller-timers-pwm-timers/>. [Accessed 24 11 2024].
- [4] [Online]. Available: <https://www.geeksforgeeks.org/difference-between-analog-and-digital-processing/>.

5. Appendix

Part 1:

1.1: Blinking LED

```
#include "mbed.h"
int main()
{
    // Initialise the digital pin LED1 as an output
    DigitalOut led(LED1);
    while (true) {
        led = 1; // turn led on
        wait(0.5); // wait for 0.5 sec
        led = 0;
        wait(0.5);
    }
}
```

Task1:

```
#include "mbed.h"

BusOut leds(LED1, LED2, LED3, LED4);

int main()
{
    while (true) {

        leds = 1;
        wait(0.2);

        leds = 2;
        wait(0.2);

        leds = 4;
        wait(0.2);

        leds = 8;
        wait(0.2);
    }
}
```

1.2: LCD Display

```
#include "mbed.h"
#include "TextLCD.h"

Serial pc(USBTX, USBRX); // tx, rx
TextLCD lcd(p28, p27, p26, p25, p24, p23); // rs, e, d4-d7

int main() {
    pc.baud(115200);
    // send string from mbed to PC
    pc.printf("Hello World!\n");
    while(1) {
        if (pc.readable()){ // check if there is any data
            char c = pc.getc(); // read data
            if (c == '0'){
                lcd.printf("Write your message");
            }
            else if (c == '1'){
                lcd.printf("this is Lana");
            }
            else if (c == '2'){
                lcd.printf("this is Aya");
            }
            else if (c == '3'){
                lcd.printf("this is Yasoomi");
            }
        }
    }
}
```

Part 2:

2.1: Reading an Analog input

<pre>:Program written using C++ for mbed LPC1114 board ; Analog input given at GPIO19 ;Output seen at all 4 onboard LEDs #include "mbed.h" BusOut ledout(LED1, LED2, LED3, LED4); AnalogIn ain(p19); int main() { float c = ain.read(); while(1){ c= ain.read(); if (c<0.1) { ledout = 0; } else if (c<0.2) { ledout = 1; } else if (c<0.3) { ledout = 2; } else if (c<0.4) { ledout = 3; } else if (c<0.5) { ledout = 4; } else if (c<0.6) { ledout = 5; } else if (c<0.7) { ledout = 6; } else if (c<0.8) { ledout = 7; } else if (c<0.9) { ledout = 8; } else if (c<1.0) { ledout = 9; } } }</pre>

Figure 6:Code of appendix 2.1

2.2: PWM-OUT:

```
#include "mbed.h"

#include "TextLCD.h"

PwmOut myled(LED1);

TextLCD lcd(p28, p27, p26, p25,
p24, p23); // rs, e, d4-d7

int main() {

while (1){

lcd.cls();

myled = myled + 0.10;

lcd.printf("LED is now
%.2F", myled.read());

wait(0.2);

if(myled == 1.0){

myled = 0;

}

}

}
```

2.3: Using Timeout Interface

```
#include "mbed.h"

Timeout flipper;

DigitalOut led1(LED1);

DigitalOut led2(LED2);

void flip() {

led2 = !led2;

}

int main() {

led2 = 1;

flipper.attach(&flip, 2.0); // setup flipper to
call flip after 2 seconds

// spin in a main loop. flipper will interrupt
it to call flip

while(1) {
```

```
led1 = !led1;

wait(0.2);

}

}
```

2.4: Using Ticker Interface

```
#include "mbed.h"

Ticker flipper;

DigitalOut led1(LED1);

DigitalOut led2(LED2);

void flip() {

led2 = !led2;

}

int main() {

led2 = 1;

flipper.attach(&flip, 2.0); // the address of
the function to be attached (flip) and the interval
(2 seconds)

// spin in a main loop. flipper will interrupt
it to call flip

while(1) {

led1 = !led1;

wait(0.2);

}
```